Exercise No: 3	Encrypting Data at Rest by Using AWS KMS
Date of Exercise	04.09.2026

AIM:

To understand and implement data encryption at rest using AWS Key Management Service (AWS KMS). To create a customer-managed encryption key, encrypt objects stored in Amazon S3 and root volumes attached to Amazon EC2 instances, monitor key usage through AWS CloudTrail, and examine the effects of disabling and re-enabling an encryption key on data accessibility.

DESCRIPTION:

This lab focused on encrypting data at rest using AWS Key Management Service (AWS KMS). A customer-managed KMS key was created and used to encrypt an object stored in Amazon S3 through server-side encryption with AWS KMS (SSE-KMS). The encryption settings were examined, and public and authenticated access to the encrypted object was tested to understand the authentication requirements for accessing KMS-encrypted data.

The activities also involved examining encryption and decryption operations through AWS CloudTrail event history. The root EBS volume of an existing EC2 instance was encrypted by creating a snapshot and generating a new encrypted volume using the customer-managed key. Finally, the KMS key was temporarily disabled to observe its effect on accessing encrypted S3 objects and starting the EC2 instance. The key was then re-enabled to restore access.

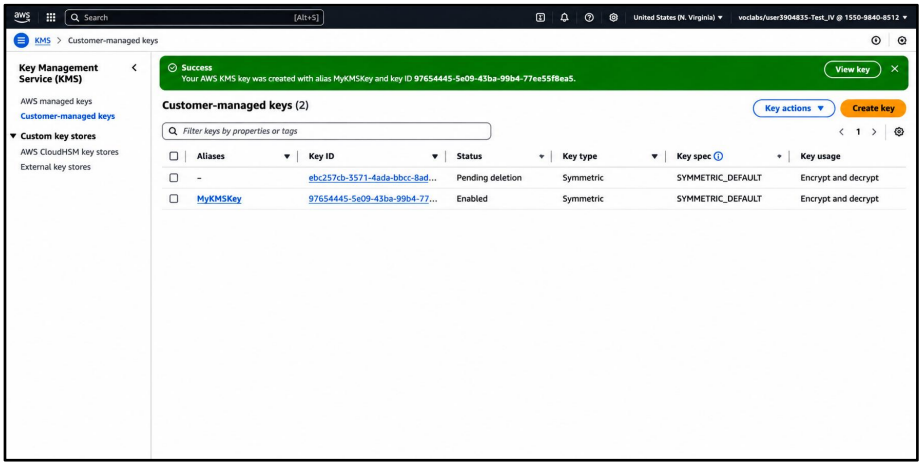
PROCEDURE:

Task 1: Creating an AWS KMS key

A customer-managed AWS KMS key was created using the AWS Key Management Service console. The key type was selected as Symmetric, and the alias MyKMSKey was assigned. The voclabs role was configured as both the key administrator and key user, providing the required permissions to manage and use the encryption key.

Result: The symmetric customer-managed KMS key named MyKMSKey was successfully created.

OUTPUT:

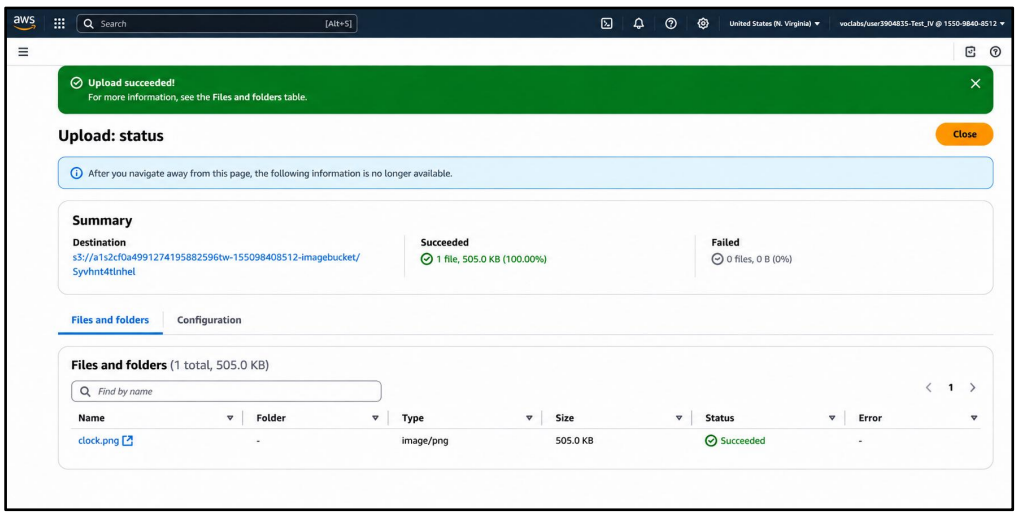


Task 2: Storing an encrypted object in an S3 bucket

The preconfigured Amazon S3 bucket whose name contained imagebucket was accessed, and its default encryption settings were examined. The clock.png image was uploaded to the bucket with server-side encryption enabled using AWS KMS (SSE-KMS). The customer-managed key MyKMSKey was selected as the encryption key. After the upload was completed, the object's properties were inspected to verify the encryption configuration.

Result: The clock.png object was successfully uploaded to the S3 bucket and encrypted using the MyKMSKey customer-managed KMS key.

OUTPUT:



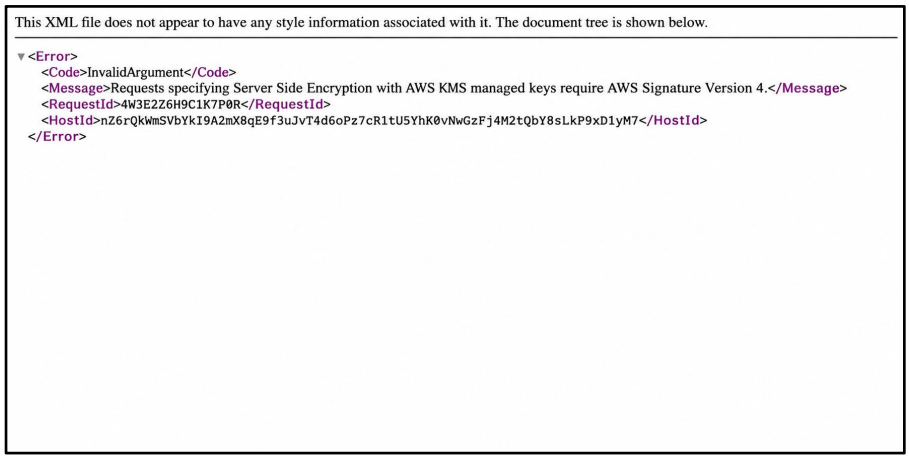
Task 3: Attempting public access to the encrypted object

The S3 object URL of clock.png was copied and opened in a new browser tab to test public access. Initially, the object returned an Access Denied error because public access was blocked at the bucket level. The bucket's public access settings were then modified, ACLs were enabled, and public read access was granted to the object. After refreshing the object

URL, an Invalid Argument error appeared instead of the previous Access Denied error. This demonstrated that although public access was granted, the encrypted object could not be decrypted through an unsigned request because SSE-KMS requires AWS Signature Version 4 authentication.

Result: Public access permissions were successfully modified, but the encrypted object remained inaccessible through an unsigned public request due to the AWS KMS authentication requirement.

OUTPUT:



Task 4: Attempting signed access to the encrypted object

The encrypted clock.png object was opened directly through the Amazon S3 console using an authenticated AWS account session. The image was successfully displayed in a new browser tab. The generated object URL was examined, showing authentication parameters associated with AWS Signature Version 4. This demonstrated how Amazon S3 authenticated the request and used AWS KMS to decrypt the encrypted data key with MyKMSKey before decrypting and displaying the image.

Result: The encrypted S3 object was successfully accessed through an authenticated request, demonstrating the decryption process using AWS KMS and Signature Version 4.

OUTPUT:

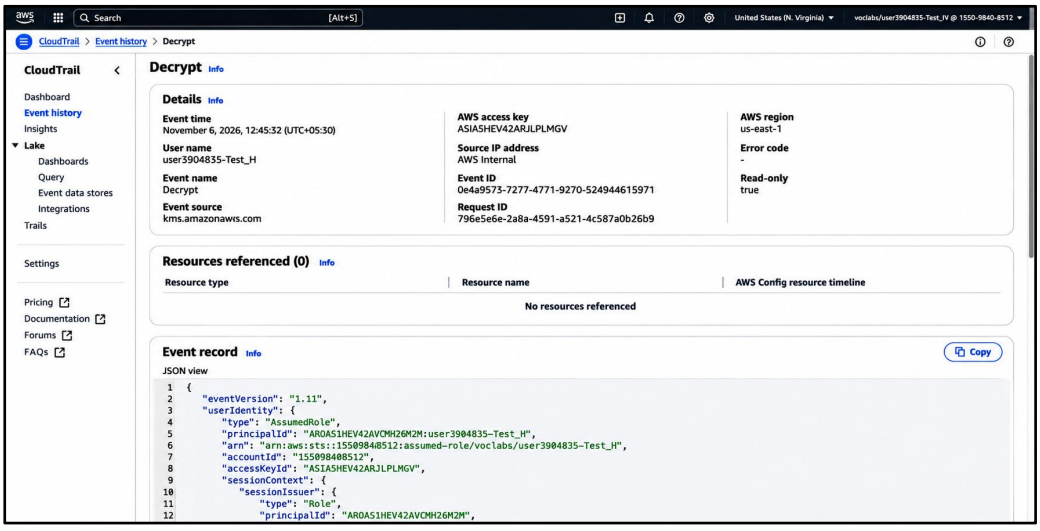


Task 5: Monitoring AWS KMS activity by using CloudTrail

The AWS CloudTrail console was accessed to examine the event history and monitor AWS KMS activity. The event history was filtered using the event source kms.amazonaws.com to identify events related to encryption operations. The GenerateDataKey event was inspected to identify the KMS key ID, encryption context, and identity responsible for generating the data key during the S3 object upload. The Decrypt event was also examined to understand how AWS KMS decrypted the encrypted data key when the clock.png object was opened through the S3 console.

Result: AWS KMS encryption and decryption activities were successfully identified and examined through CloudTrail event history.

OUTPUT:



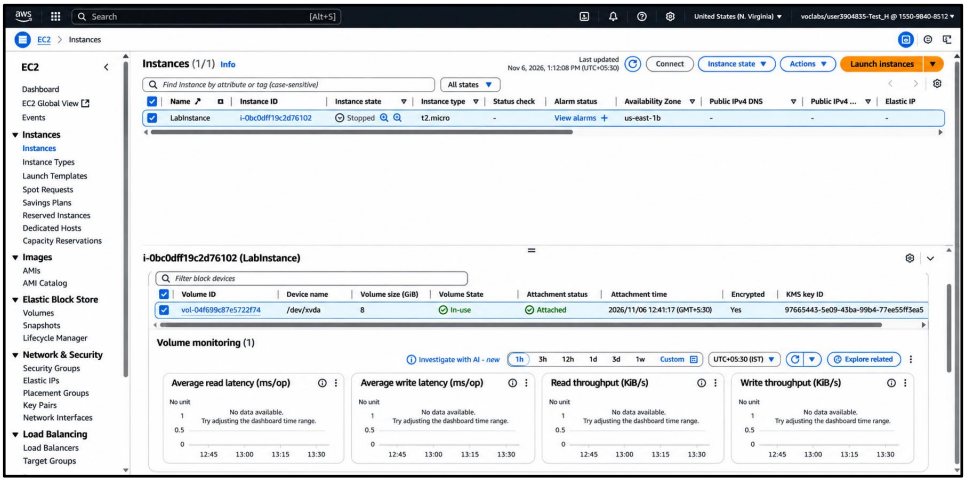
Task 6: Encrypting the root volume of an existing EC2 instance

The LabInstance EC2 instance was stopped, and a snapshot of its existing unencrypted root EBS volume was created. After the snapshot reached the Completed state, a new encrypted volume was created from it with encryption enabled and MyKMSKey selected. The original volume was identified as the old unencrypted root volume, while the newly created volume was identified as the new encrypted root volume. The old volume was then detached, and the

encrypted volume was attached to LabInstance using the device name /dev/xvda. The instance storage settings were inspected to verify that the attached root volume was encrypted and associated with MyKMSKey.

Result: The unencrypted root volume was successfully replaced with a new EBS volume encrypted using the MyKMSKey customer-managed KMS key.

OUTPUT:

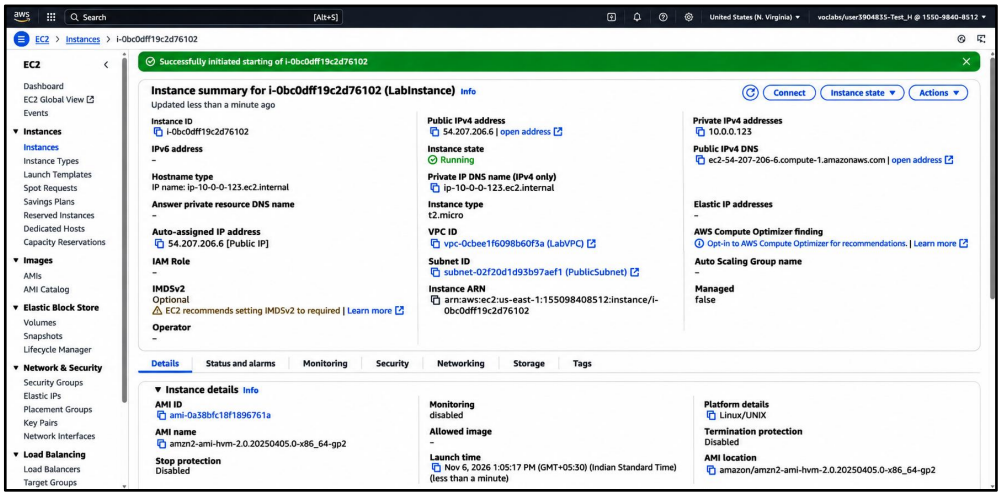


Task 7: Disabling the encryption key and observing the effects

The customer-managed KMS key named MyKMSKey was temporarily disabled through the AWS KMS console. The LabInstance EC2 instance was then started to observe the effect of disabling the encryption key. The instance failed to reach the Running state and returned to Stopped because AWS KMS could not decrypt the data key required to access the encrypted root volume. The encrypted clock.png object was also opened through Amazon S3, resulting in a KMS.DisabledException error. The corresponding DisableKey, StartInstances, and CreateGrant events were examined in CloudTrail to understand the failure. Finally, MyKMSKey was re-enabled, and the LabInstance was started again successfully.

Result: The effects of disabling the KMS key on encrypted S3 and EBS resources were successfully observed. Re-enabling the key restored access to the encrypted object and allowed the EC2 instance to start.

OUTPUT:



BADGE:



PRESENTATION LINK:

[Insert YouTube / presentation link]

RESULT:

The experiment demonstrated data encryption at rest using a customer-managed AWS KMS key. An Amazon S3 object was encrypted using SSE-KMS, and access to the encrypted object was tested. CloudTrail events were examined to understand encryption-related operations. The root EBS volume of an existing EC2 instance was also encrypted using the KMS key. Disabling the key demonstrated its effect on access to encrypted resources, while re-enabling it restored access.